

Web Backend Fundamentals

Spring Boot Training Program | Week 1

LECTURE NOTES

Part 1: Core Architecture

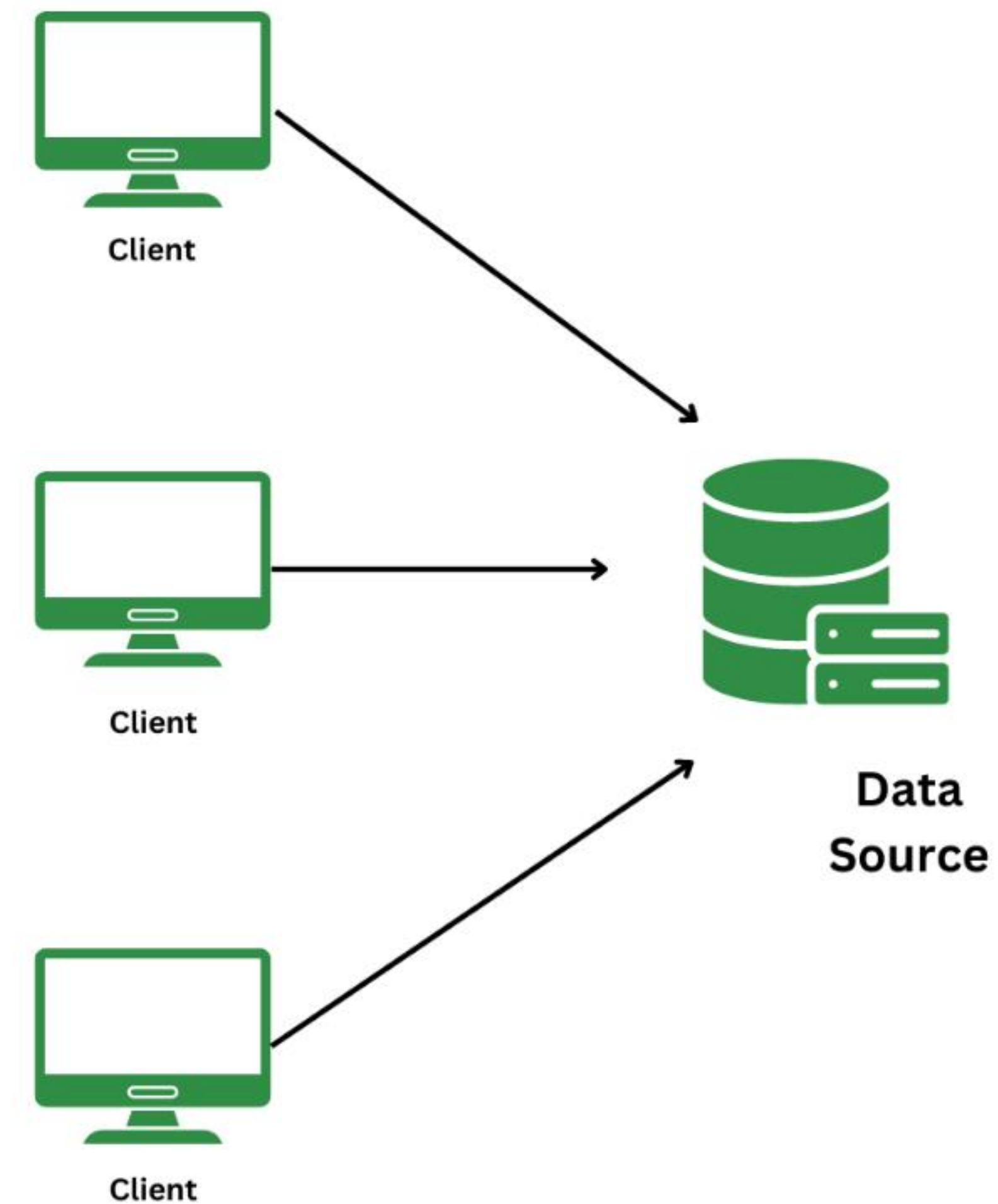
Understanding how the modern web connects devices and data through the client-server model.

The Client-Server Model

The **Client-Server Architecture** is the backbone of the web. It separates user interaction from data processing.

-  **The Client:** Browsers (Chrome/Safari) or Apps that initiate requests for data.
-  **The Server:** Spring Boot applications that listen for requests and execute logic.
-  **The Database:** Persistent storage that holds system information (SQL/NoSQL).

Two Tier Architecture



HTTP Verbs: The Action Center

In backend development, we use specific HTTP methods to tell the server exactly what operation we want to perform.



GET

Retrieves a resource.
It is **Read-Only** and should never modify data.



POST

Submits data to the server to **Create** a new resource.



PUT / PATCH

Used to **Update** existing data. PUT replaces, PATCH modifies partially.



DELETE

Instructs the server to **Remove** a specific resource.

Anatomy of an HTTP Request

1. The Endpoint & Method

Every request needs a **URL** (The Destination) and a **Method** (The Intent).

```
GET /api/v1/users/42
```

2. Headers & Body

Headers contain metadata (Content-Type, Tokens).
The **Body** contains actual data for POST/PUT requests.

```
Content-Type: application/json
```

HTTP Status Codes

Category	Meaning	Common Examples
2xx Success	The request worked!	200 OK, 201 Created
3xx Redirection	The resource moved.	301 Permanent, 302 Found
4xx Client Error	The client did something wrong.	400 Bad Request, 404 Not Found
5xx Server Error	The server crashed.	500 Internal Error, 503 Service Unavailable

Part 2: RESTful Design

Learning the principles that make APIs predictable, scalable, and easy to integrate.

Core REST Principles



Statelessness: The server doesn't remember previous requests. Each request must contain all necessary info.



Uniform Interface: Standardized naming and methods allow any client to talk to the server.



Resource-Based: URLs represent resources (nouns like 'users') rather than actions.

"REST is not a protocol, but a set of architectural constraints for building modern APIs."

Resource Mapping

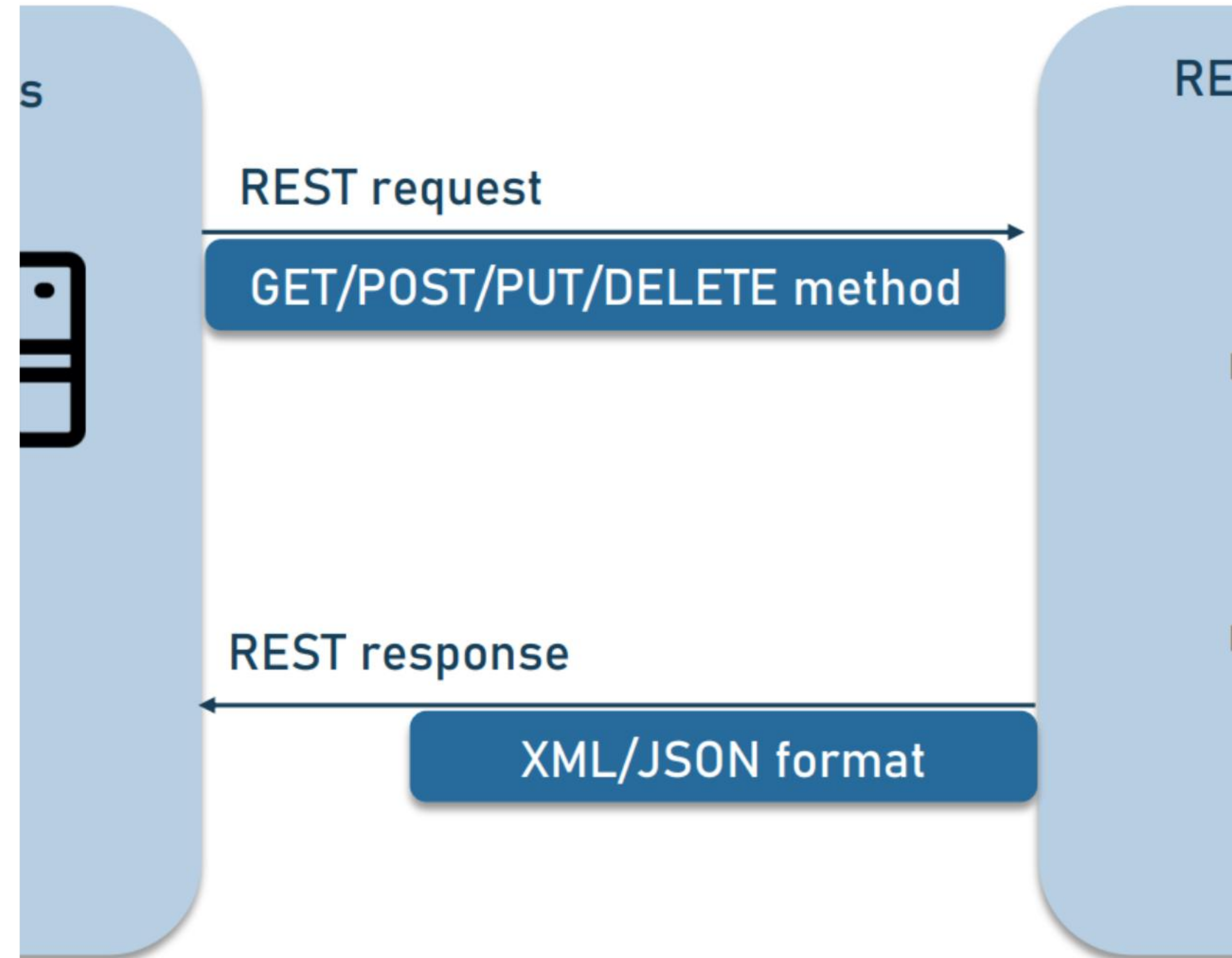
Nouns, not Verbs: RESTful URLs identify *things*, while HTTP methods identify the *action*.

✗ GET /getAllUsers

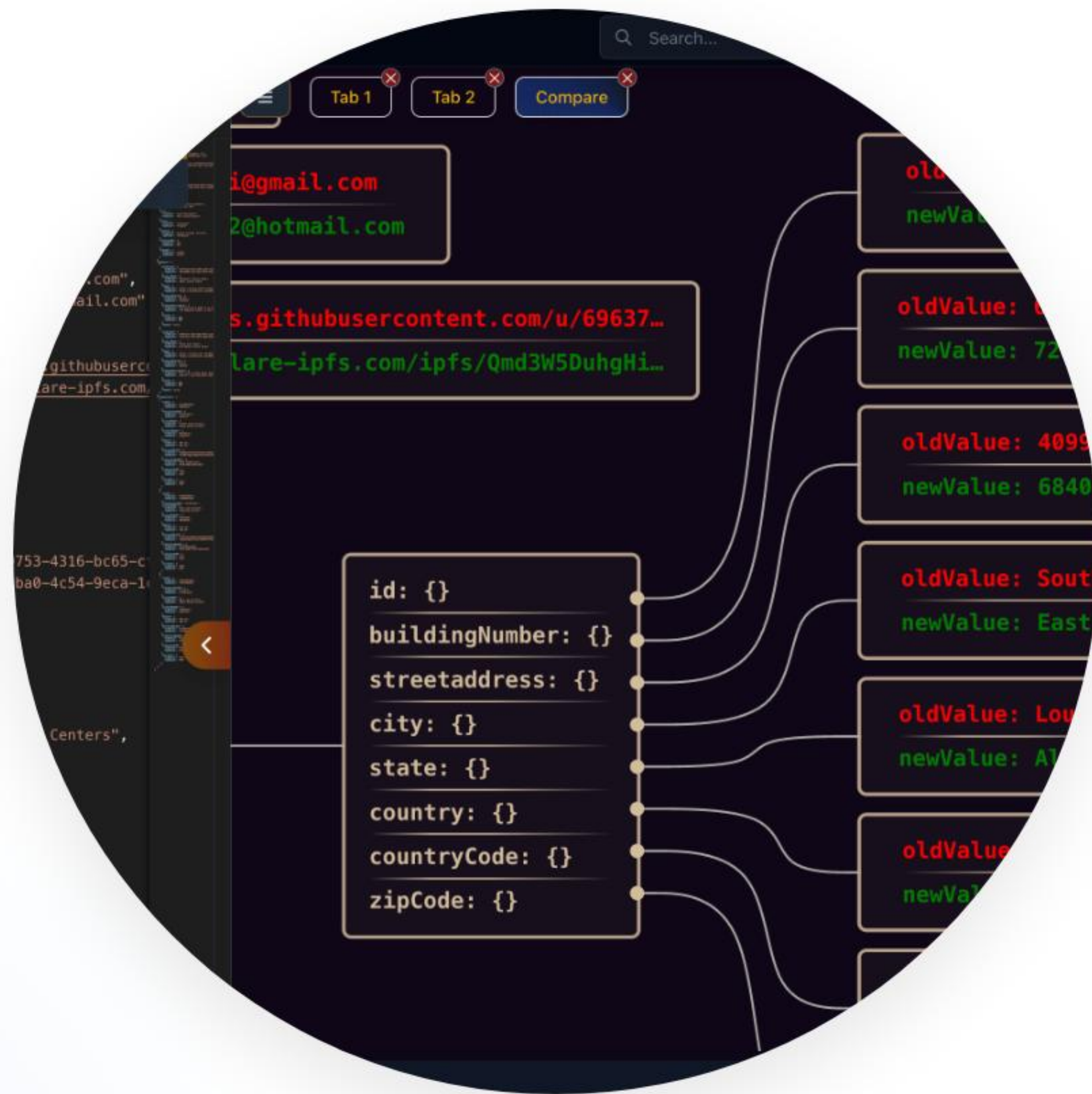
✓ GET /users

This creates a clean, predictable structure that developers worldwide can understand without extensive documentation.

REST API IN ACTION



JSON: The Language of APIs



Lightweight Data Exchange

JSON (JavaScript Object Notation) is a text-based format for storing and transporting data.

- ✓ **Simple:** Uses key-value pairs.
- ✓ **Fast:** Low overhead compared to XML.
- ✓ **Universal:** Every language (Java, JS, Python) can read it easily.

The Request Lifecycle



Questions?

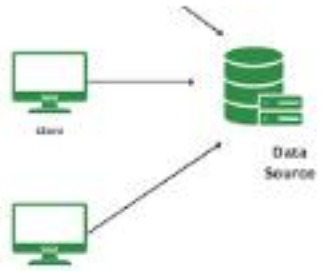
Next Up: Building our first REST Controller with Spring Boot

Web Arch
Client-Server

HTTP
Verbs & Codes

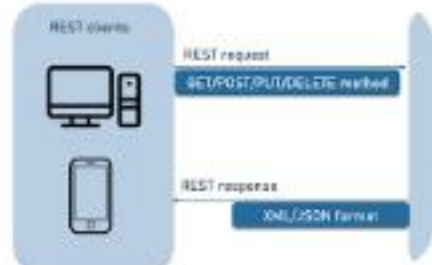
Data
JSON Format

Image Sources



<https://media.geeksforgeeks.org/wp-content/uploads/20240907155154/2.png>

Source: www.geeksforgeeks.org



https://www.altexsoft.com/media/2021/03/rest_api_works.png

Source: www.altexsoft.com



<https://jsonviewer.tools/features/compare.webp>

Source: jsonviewer.tools